ЛАБОРАТОРНА РОБОТА № 2

ПОРІВНЯННЯ МЕТОДІВ КЛАСИФІКАЦІЇ ДАНИХ

Мета: використовуючи спеціалізовані бібліотеки та мову програмування Python дослідити різні методи класифікації даних та навчитися їх порівнювати.

Хід роботи:

Завдання 1: Класифікація за допомогою машин опорних векторів (SVM).

Створіть класифікатор у вигляді машини опорних векторів, призначений для прогнозування меж доходу заданої фізичної особи на основі 14 ознак (атрибутів). Метою є з'ясування умов, за яких щорічний прибуток людини перевищує $50000 або менше цієї величини за допомогою бінарної класифікації.

№	Назва ознаки (Feature Name)	Опис ознаки	Тип ознаки
1	age	Вік особи	Числовий (Continuous)
2	workclass	Тип/сектор зайнятості (напр., Private, Self-emp-not-inc, State-gov)	Категоріальний (Categorical)
3	fnlwgt	Фінальна вага вибірки (статична вага, розрахована Бюро перепису)	Числовий (Continuous)
4	education	Найвищий рівень освіти (напр., Bachelors, HS-grad, Some-college)	Категоріальний (Categorical)
5	education-num	Кількість років навчання у числовому вигляді	Числовий (Continuous)
6	marital-status	Сімейний стан (напр., Married-civ-spouse, Never-married, Divorced)	Категоріальний (Categorical)
7	occupation	Род/сфера діяльності (напр., Tech-support, Exec-managerial, Craft-repair)	Категоріальний (Categorical)
8	relationship	Відносини в сім'ї (напр., Wife, Own-child, Husband, Not-in-family)	Категоріальний (Categorical)

9	race	Расова приналежність (напр., White, Asian-Pac-Islander, Black)	Категоріальний (Categorical)
10	sex	Стать (Female, Male)	Категоріальний (Categorical)
11	capital-gain	Прибуток від капіталу (прибуток від продажу активів)	Числовий (Continuous)
12	capital-loss	Збитки від капіталу	Числовий (Continuous)
13	hours-per-week	Кількість робочих годин на тиждень	Числовий (Continuous)
14	native-country	Країна походження (напр., United-States, Mexico, Canada)	Категоріальний (Categorical)
-	income (Цільова мітка)	Рівень річного доходу: <=50K або >50K	Категоріальний (Цільовий клас)

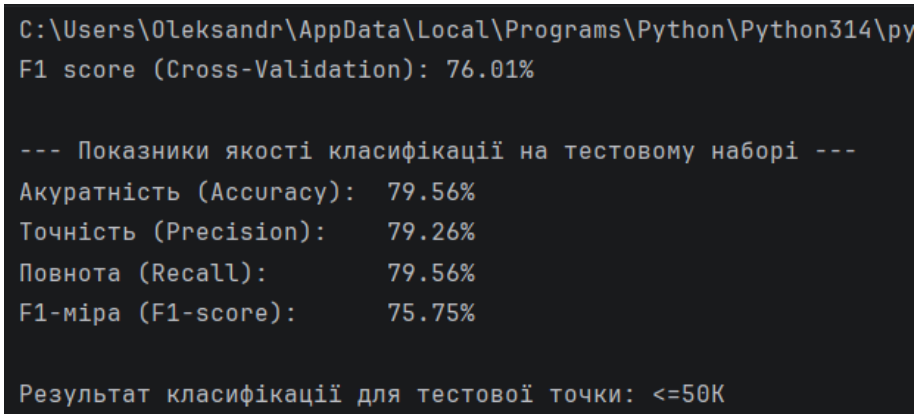


Рисунок 1. Результат виконання програми

Висновок щодо тестової точки:

Вхідні дані точки: 37 років, приватний сектор (Private), освіта HS-grad (середня освіта, 9 років), невідокремлена категорія праці (Handlers-cleaners), 40 годин/тиждень, без капіталовкладень (capital-gain=0, capital-loss=0).

Аналітичний висновок: Класифікатор на основі машин опорних векторів з лінійним ядром навчився прогнозувати, що даний робітник із середньою освітою та стандартним робочим тижнем без додаткового прибутку від капіталу отримує річний дохід менше або дорівнює $50,000 на рік.

Лістинг програми представлено в додатку А.

Завдання 2: Порівняння якості класифікаторів SVM з нелінійними ядрами

1. **Поліноміальне ядро.**

```
=== Поліноміальне ядро (Polynomial Kernel) ===
Акуратність (Accuracy):  61.25%
Точність (Precision):    74.52%
Повнота (Recall):        61.25%
F1-міра (F1-score):      51.04%
```

Рисунок 2.1. Поліноміальне ядро

Під час тренування SVM з поліноміальним ядром на повному наборі даних (max_datapoints = 25000) алгоритм демонстрував надзвичайно високу обчислювальну складність, що призводило до «зависання» програми та неможливості завершити обчислення за розумний час. Для забезпечення збіжності алгоритму розмір вибірки для поліноміального ядра було вимушено зменшено до max_datapoints = 10000 (сумарно 20 000 точок даних). Скорочення обсягу навчальних даних очікувано знизило узагальнюючу здатність моделі, через що поліноміальне ядро отримало найнижче значення F1-міри (51.04%).

Поліноміальне ядро через обмеження вибірки отримало Accuracy 61.25%, однак показало високий рівень Precision (74.52%), що свідчить про низьку кількість хибнопозитивних спрацьовувань, але низьку повноту охоплення другого класу.

2. **Гаусове ядро.**

```
=== Гаусове ядро (RBF Kernel) ===
Акуратність (Accuracy):  78.19%
Точність (Precision):    82.82%
Повнота (Recall):        78.19%
F1-міра (F1-score):      71.51%
```

Рисунок 2.2. Гаусове ядро

3. Сигмоїдальне ядро.

```
=== Сигмоїдальне ядро (Sigmoid Kernel) ===
Акуратність (Accuracy):  60.47%
Точність (Precision):    60.64%
Повнота (Recall):        60.47%
F1-міра (F1-score):      60.55%
```

Рисунок 2.3. Сигмоїдальне ядро

За результатами проведених експериментів найкращим класифікатором для набору даних є SVM з Гаусовим ядром (rbf). Воно забезпечує оптимальний баланс між точністю прогнозування доходу та обчислювальною складністю на повній вибірці даних.

Лістинг програм представлено в додатку Б.

Завдання 3: Порівняння якості класифікаторів на прикладі класифікації сортів ірисів

```
Ключі iris_dataset:
 dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])

Короткий опис (DESCR):
 .. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive
...

Назви відповідей (target_names):
 ['setosa' 'versicolor' 'virginica']
Назви ознак (feature_names):
 ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Тип масиву data: <class 'numpy.ndarray'>
Форма масиву data: (150, 4)
Перші 5 елементів data:
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
Тип масиву target:<class 'numpy.ndarray'>
Масив target:
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]
```

Рисунок 3.1. Структура даних

```
Перші 20 рядків (head):
     sepal-length  sepal-width  petal-length  petal-width        class
0             5.1          3.5           1.4          0.2  Iris-setosa
1             4.9          3.0           1.4          0.2  Iris-setosa
2             4.7          3.2           1.3          0.2  Iris-setosa
3             4.6          3.1           1.5          0.2  Iris-setosa
4             5.0          3.6           1.4          0.2  Iris-setosa
5             5.4          3.9           1.7          0.4  Iris-setosa
6             4.6          3.4           1.4          0.3  Iris-setosa
7             5.0          3.4           1.5          0.2  Iris-setosa
8             4.4          2.9           1.4          0.2  Iris-setosa
9             4.9          3.1           1.5          0.1  Iris-setosa
10            5.4          3.7           1.5          0.2  Iris-setosa
11            4.8          3.4           1.6          0.2  Iris-setosa
12            4.8          3.0           1.4          0.1  Iris-setosa
13            4.3          3.0           1.1          0.1  Iris-setosa
14            5.8          4.0           1.2          0.2  Iris-setosa
15            5.7          4.4           1.5          0.4  Iris-setosa
16            5.4          3.9           1.3          0.4  Iris-setosa
17            5.1          3.5           1.4          0.3  Iris-setosa
18            5.7          3.8           1.7          0.3  Iris-setosa
19            5.1          3.8           1.5          0.3  Iris-setosa

Статистичне зведення (describe):
        sepal-length  sepal-width  petal-length  petal-width
count     150.000000   150.000000    150.000000   150.000000
mean        5.843333     3.054000      3.758667     1.198667
std         0.828066     0.433594      1.764420     0.763161
min         4.300000     2.000000      1.000000     0.100000
25%         5.100000     2.800000      1.600000     0.300000
50%         5.800000     3.000000      4.350000     1.300000
75%         6.400000     3.300000      5.100000     1.800000
max         7.900000     4.400000      6.900000     2.500000

Розподіл за класами:
 class
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
dtype: int64
```

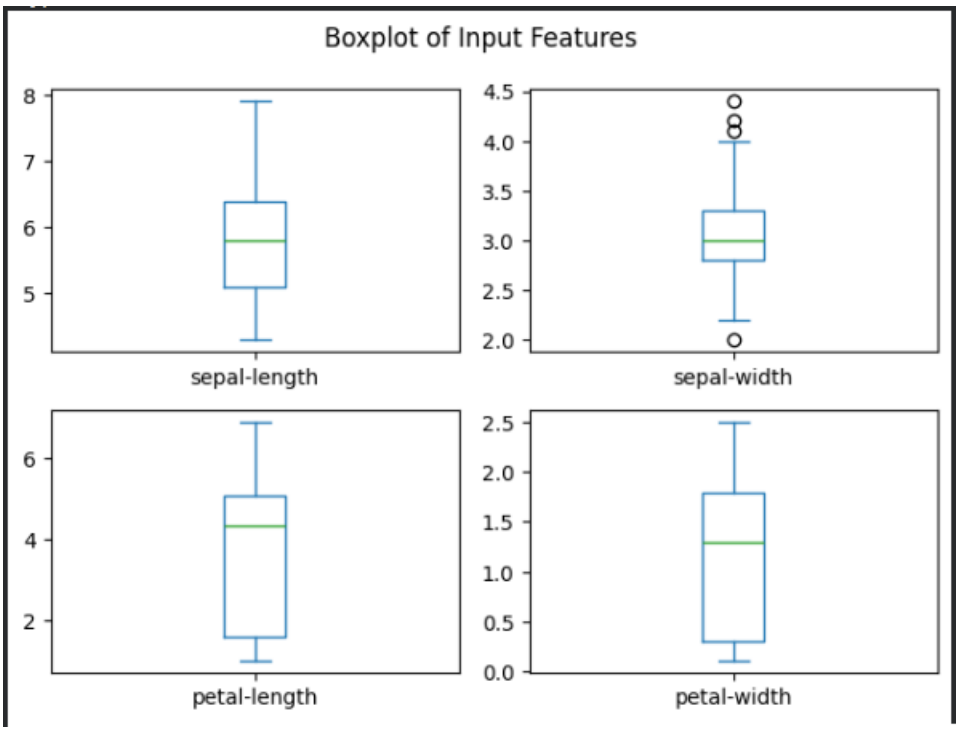


Рисунок 3.2. Діаграма розмаху

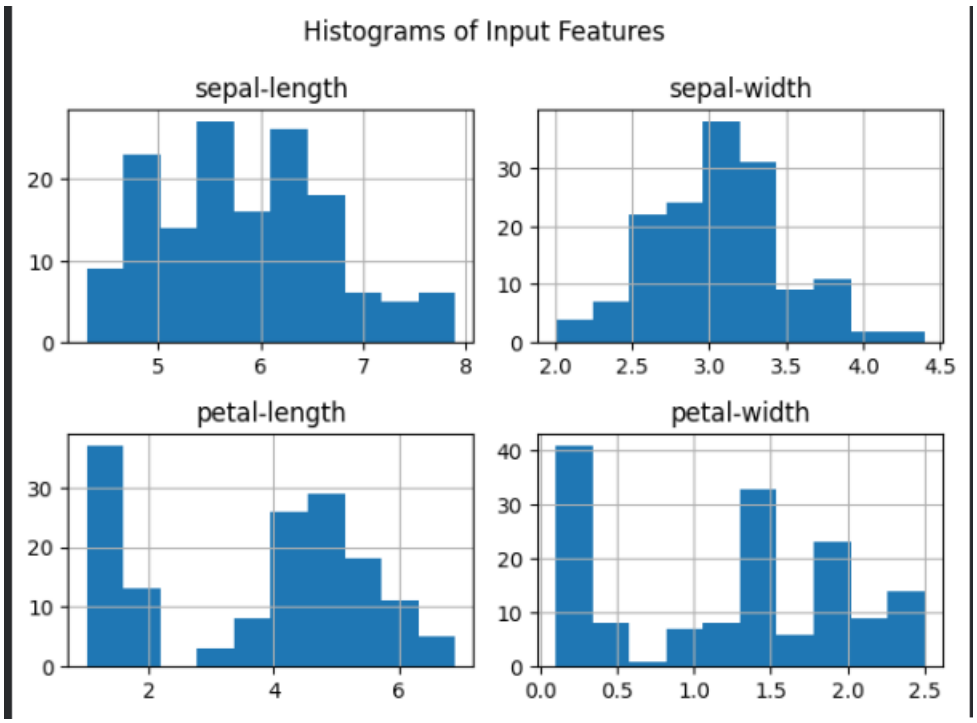


Рисунок 3.3. Гістограма розподілу атрибутів датасета

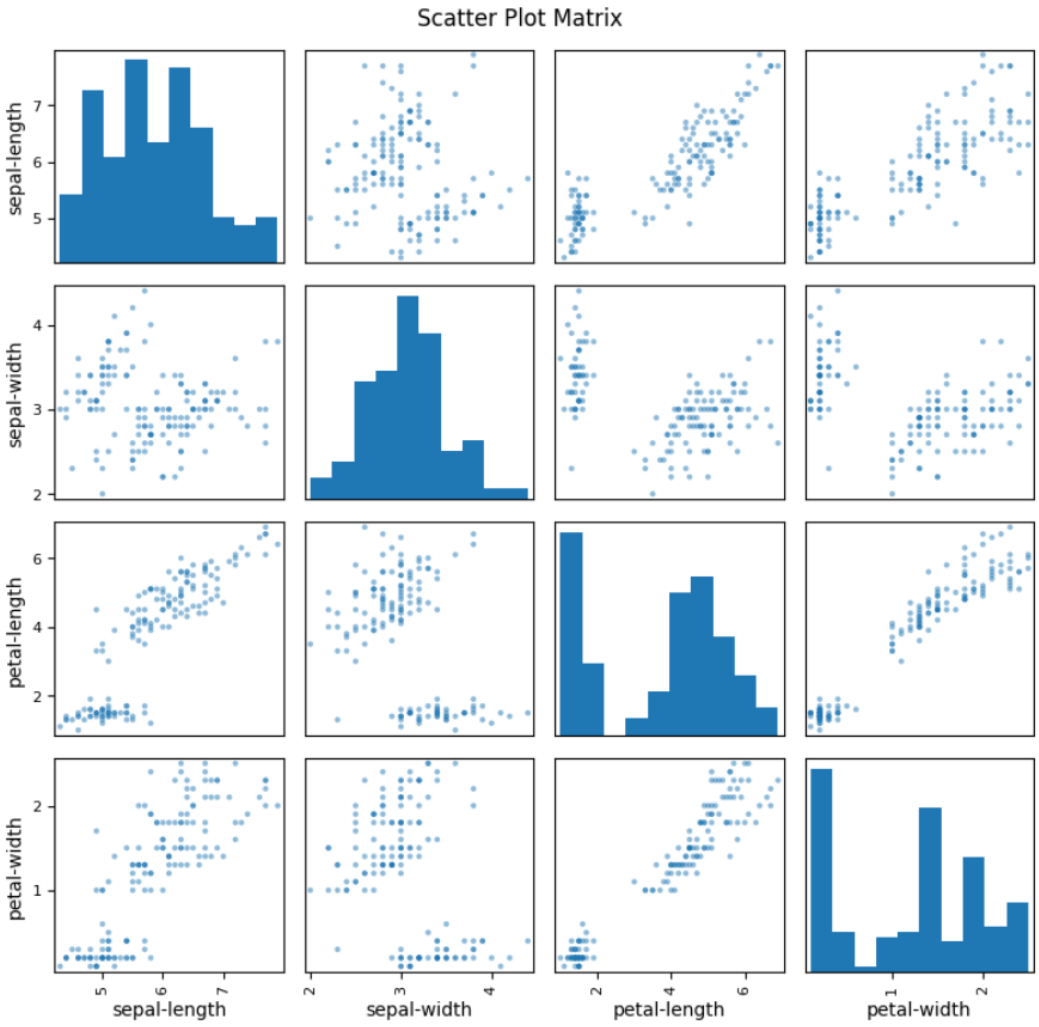


Рисунок 3.4. Матриця діаграм розсіювання

```
--- Результати 10-кратної стратифікованої крос-валідації ---
LR: 0.941667 (0.065085)
LDA: 0.975000 (0.038188)
KNN: 0.958333 (0.041667)
CART: 0.933333 (0.050000)
NB: 0.950000 (0.055277)
SVM: 0.983333 (0.033333)
```

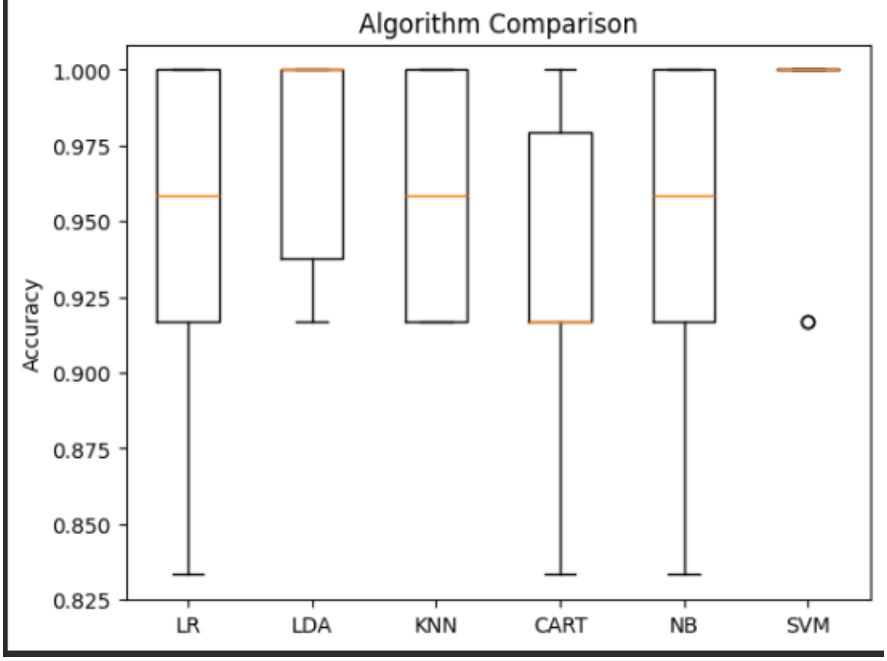


Рисунок 3.5. Порівняння алгоритмів

Найкращі результати під час 10-кратної стратифікованої крос-валідації продемонстрували класифікатори SVM (середня точність 98.33%) та LDA (97.50%).

Метод SVM обрано як найкращий, оскільки він будує оптимальну розділювальну гіперплощину з максимальним зазором між класами в багатовимірному просторі ознак, що гарантує високу узагальнюючу здатність моделі та мінімальне стандартне відхилення (0.033).

```
--- Оцінка точності на контрольні вибірці (Validation Set) ---
Accuracy Score: 0.9666666666666667

Confusion Matrix:
 [[11  0  0]
 [ 0 12  1]
 [ 0  0  6]]

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        11
Iris-versicolor       1.00      0.92      0.96        13
 Iris-virginica       0.86      1.00      0.92         6

       accuracy                           0.97        30
      macro avg       0.95      0.97      0.96        30
   weighted avg       0.97      0.97      0.97        30


Форма масиву X_new: (1, 4)
Прогноз (клас): Iris-setosa
```

Модель досягла точності 96.67% на контрольних даних (X_validation), допустивши лише 1 помилку класифікації між класами versicolor та virginica.

Для об'єкта з параметрами (довжина чашолистка = 5 см, ширина = 2.9 см, довжина пелюстки = 1 см, ширина пелюстки = 0.2 см) модель передбачила клас Iris-setosa.

Обґрунтування: Малі розміри пелюсток (довжина 1 см, ширина 0.2 см) є характерною діагностичною ознакою саме сорту Iris setosa (що видно з одномірних та двомірних графіків розподілу).

Лістинг програмного коду наведено в додатку В.

Завдання 4: Порівняння якості класифікаторів для набору даних завдання 2.1.

Для оптимізації швидкості обчислень та забезпечення стабільної збіжності моделей (особливо для SVM та KNN) обсяг вибірки встановлено у розмірі max_datapoints = 10000 (по 10,000 записів для кожного класу, сумарно 20,000 точок).

```
LR: 0.768742 (0.009030)
LDA: 0.749821 (0.011171)
KNN: 0.775168 (0.013607)
CART: 0.757531 (0.011381)
NB: 0.713980 (0.006948)
SVM: 0.807796 (0.011327)
```

Рисунок 4.1. Результати 10-кратної крос-валідації

Алгоритм	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
LR	77.81	77.75	77.81	77.68
LDA	76.13	76.03	76.13	76.01
KNN	78.75	78.75	78.75	78.75
CART	76.96	76.93	76.96	76.94
NB	71.33	74.67	71.33	68.95
SVM	81.84	82.03	81.84	81.89

Рисунок 4.2. Показники якості

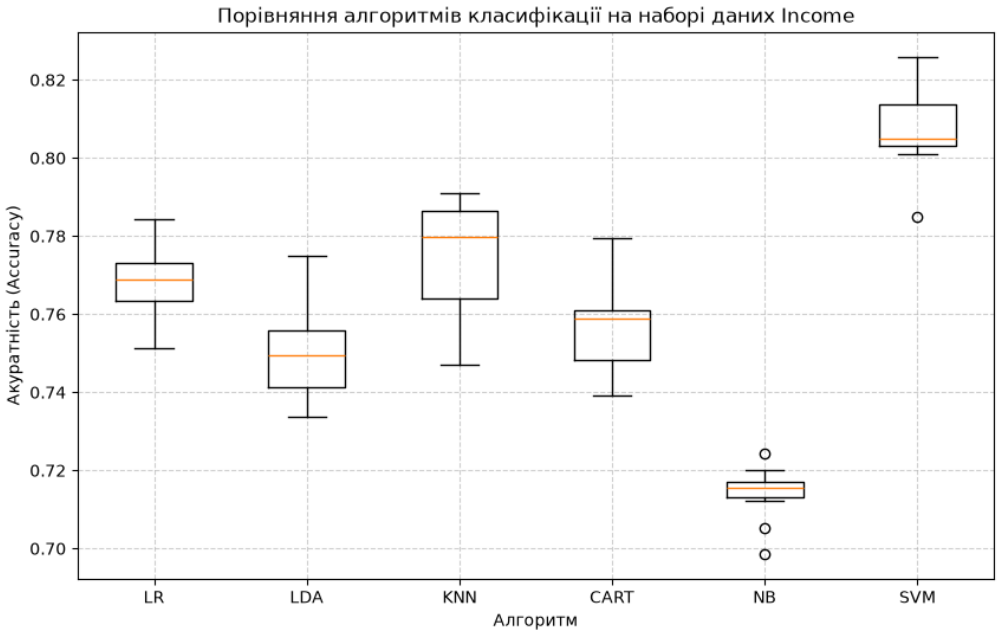


Рисунок 4.3. Порівняння алгоритмів класифікації

Найкращі результати за всіма метриками продемонстрував алгоритм SVM. На етапі 10-кратної стратифікованої крос-валідації він досяг середньої точності 80.78%, а на тестовій вибірці — Accuracy 81.84% та F1-Score 81.89%.

Другим за ефективністю виявився метод k-найближчих сусідів (KNN), який отримав 78.75% за всіма метриками якості на тестовій вибірці (та 77.52% на крос-валідації).

Лінійні класифікатори — Логістична регресія (LR) та Лінійний дискримінантний аналіз (LDA) — показали стабільні серединні результати з F1-score 77.68% та 76.01% відповідно.

Найнижчу якість розпізнавання отримано для Наївного баєсовського класифікатора (NB): акуратність склала 71.33%, а F1-міра знизилася до 68.95%.

Ефективність на комбінованих даних: Набір даних income_data.txt містить як числові, так і категоріальні ознаки. Метод опорних векторів (SVM) найкраще будує нелінійну розділювальну межу між класами доходів (<=50K та >50K) у багатовимірному просторі.

Збалансованість метрик: У моделей SVM та KNN спостерігається висока точність (Precision 82.03%) та повнота (Recall 81.84%), що свідчить про відсутність викривлень у бік більшого за кількістю класу та високу узагальнюючу здатність моделей.

Для розв'язання задачі бінарної класифікації рівнів доходів населення за набором даних income_data.txt найкращим обрано метод опорних векторів (SVM). Він забезпечує найвищу точність прогнозування (понад 81.8%) та мінімальну кількість помилок порівняно з іншими 5 алгоритмами.

Лістинг програмного коду наведено в додатку Г.

Завдання 5: Класифікація даних лінійним класифікатором Ridge.

RidgeClassifier перетворює цільові мітки на значення $\{-1,1\}$ і розв'язує задачу за допомогою лінійної регресії з L_2-регуляризацією (регресія Ріджа).

- tol=1e-2 (tolerance — допуск): Гранична точність для зупинки обчислювального алгоритму. Градієнтний метод припиняє ітерації, коли зміна функції втрат стає меншою за 0.01 (10^{-2}).
- solver="sag" (Stochastic Average Gradient — стохастичний усереднений градієнтний спуск): Метод оптимізації. Він забезпечує швидку збіжність при пошуку мінімуму функції втрат із L_2-регуляризацією.

```
--- Показники якості класифікатора Ridge ---
Accuracy: 0.7556
Precision: 0.8333
Recall: 0.7556
F1 Score: 0.7503
Cohen Kappa Score: 0.6431
Matthews Corrcoef: 0.6831

Classification Report:

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       0.89      0.44      0.59        18
   virginica       0.50      0.91      0.65        11

    accuracy                           0.76        45
   macro avg       0.80      0.78      0.75        45
weighted avg       0.83      0.76      0.75        45
```

Рисунок 5.1. Результат виконання програми

На основі результатів тестування модель отримала такі показники:

- Accuracy (75.56%): Загальна частка правильно класифікованих об'єктів (34 правильно розпізнані квітки з 45).
- Precision (83.33%): Зважена точність класифікатора, яка вимірює частку дійсно правильних об'єктів серед усіх, віднесених моделлю до відповідного класу.
- Recall (75.56%): Зважена повнота, що показує частку знайдених об'єктів кожного класу від їх загальної реальної кількості.
- F1 Score (75.03%): Зважене гармонійне середнє між Precision та Recall.

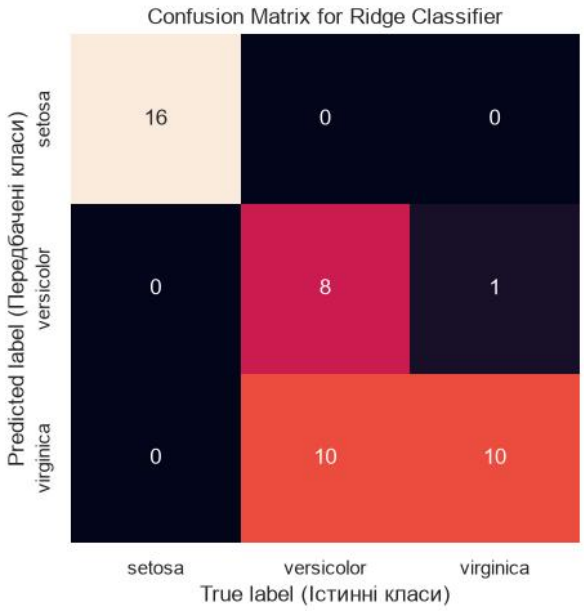


Рисунок 5.2. Confusion

Аналіз матриці помилок за графіком (де по осі *X* розміщено істинні класи True label, а по осі *Y* — передбачені Predicted label):

Клас setosa: 16 з 16 об'єктів класифіковано бездоганно (100% точність та повнота).

Клас versicolor: З 18 істинно існуючих квітів модель правильно визначила лише 8, тоді як 10 були помилково віднесені до класу virginica. Це пояснює низький параметр Recall (0.44) для цього класу.

Клас virginica: З 11 істинно існуючих квітів модель правильно розпізнала 10, а 1 помилково віднесла до versicolor. Однак через те, що модель також «приписала» до virginica 10 квітів класу versicolor, точність (Precision) для цього класу впала до 0.50.

Лінійний класифікатор Ridge легко відокремлює клас setosa, оскільки він є лінійно роздільним. Проте через лінійний характер межі прийняття рішень модель не здатна чітко розмежувати класи versicolor та virginica, які перекриваються між собою у просторі ознак.

Коефіцієнт Каппа Коена (Cohen Kappa Score = 0.6431):

- Що оцінює: Метрика вимірює рівень узгодженості між передбаченими та істинними мітками з урахуванням ймовірності випадкового вгадування.
- Результат: Значення 0.6431 належить до діапазону $0.61 - 0.80$, що відповідає добрій / суттєвій узгодженості (Substantial Agreement) прогнозу класифікатора з реальним розподілом класів.

Коефіцієнт кореляції Метьюза (Matthews Corrcoef = 0.6831):

- Що оцінює: Враховує всі чотири елементи матриці плутанини (True/False Positives та Negatives) для комплексної оцінки якості мультикласової класифікації.
- Результат: Значення 0.6831 підтверджує наявність стійкого позитивного зв'язку між передбаченнями Ridge-класифікатора та реальними значеннями.

Лістинг програмного коду викладено в додатку Д.

Посилання на GitHub: https://github.com/kn232bov/AI-systems.

Висновок: використовуючи спеціалізовані бібліотеки та мову програмування Python дослідив різні методи класифікації даних та навчився їх порівнювати.

Додаток А:

```
import numpy as np
from sklearn import preprocessing
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsOneClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Вхідний файл з даними
input_file = 'income_data.txt'
X = []
y = []
count_class1 = 0
count_class2 = 0
max_datapoints = 25000
# Читання та первинна обробка даних
with open(input_file, 'r') as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break
        # Пропуск порожніх або битих рядків
        if '?' in line or len(line.strip()) == 0:
            continue
        # Розбиття рядка за комами
        data = line.strip().split(', ')
        # Перевірка та балансування класів
        if data[-1] == '<=50K' and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1
```

```
        elif data[-1] == '>50K' and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

# Перетворення на масив numpy
X = np.array(X)
# Перетворення рядкових категоріальних даних на числові
label_encoder = []
X_encoded = np.empty(X.shape, dtype=object)
for i, item in enumerate(X[0]):
    if item.isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        encoder = preprocessing.LabelEncoder()
        X_encoded[:, i] = encoder.fit_transform(X[:, i])
        label_encoder.append(encoder)

# Окремо виділяємо ознаки X та цільові мітки y
X_data = X_encoded[:, :-1].astype(int)
y_data = X_encoded[:, -1].astype(int)
# Створення та розбиття на навчальну та тестову вибірки (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_data, y_data, test_size=0.2, random_state=5
)
# Створення та навчання SVM-класифікатора з лінійним ядром
classifier = OneVsOneClassifier(LinearSVC(random_state=0, max_iter=2000))
classifier.ft(X_train, y_train)
# Прогнозування на тестовій вибірці
y_test_pred = classifier.predict(X_test)
```

```
# Обчислення F1-міри за допомогою 3-кратної крос-валідації
f1_cv = cross_val_score(classifier, X_data, y_data, scoring='f1_weighted', cv=3)
print("F1 score (Cross-Validation): " + str(round(100 * f1_cv.mean(), 2)) + "%")
# Обчислення додаткових показників якості (Метрики ЛР-1)
accuracy = accuracy_score(y_test, y_test_pred)
precision = precision_score(y_test, y_test_pred, average='weighted')
recall = recall_score(y_test, y_test_pred, average='weighted')
f1_test = f1_score(y_test, y_test_pred, average='weighted')
print("\n--- Показники якості класифікації на тестовому наборі ---")
print(f"Акуратність (Accuracy):  {round(accuracy * 100, 2)}%")
print(f"Точність (Precision):    {round(precision * 100, 2)}%")
print(f"Повнота (Recall):        {round(recall * 100, 2)}%")
print(f"F1-міра (F1-score):      {round(f1_test * 100, 2)}%\n")
# Передбачення для довільної тестової точки
input_data = [
   '37', 'Private', '215646', 'HS-grad', '9', 'Never-married',
   'Handlers-cleaners', 'Not-in-family', 'White', 'Male',
   '0', '0', '40', 'United-States'
]
# Кодування тестової точки
input_data_encoded = [-1] * len(input_data)
count = 0
for i, item in enumerate(input_data):
   if item.isdigit():
      input_data_encoded[i] = int(input_data[i])
   else:
      input_data_encoded[i] = int(label_encoder[count].transform([input_data[i]])[0])
      count += 1

input_data_encoded = np.array(input_data_encoded).reshape(1, -1)
```

```
# Передбачення класу
predicted_class = classifier.predict(input_data_encoded)
predicted_label = label_encoder[-1].inverse_transform(predicted_class)[0]
print(f"Результат класифікації для тестової точки: {predicted_label}")
```

Додаток Б:
LR_2_task_2_1.py:

```
import numpy as np
from sklearn import preprocessing
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Вхідний файл з даними
input_file = 'income_data.txt'

X = []
y = []
count_class1 = 0
count_class2 = 0
# Зменшено розмір вибірки до 10 000 для оптимізації обчислювальної складності
max_datapoints = 10000

with open(input_file, 'r') as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break
        if '?' in line or len(line.strip()) == 0:
            continue
        data = line.strip().split(', ')
        if data[-1] == '<=50K' and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1
        elif data[-1] == '>50K' and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

X = np.array(X)
```

```
# 2. Кодування категоріальних ознак
X_encoded = np.empty(X.shape, dtype=object)
for i, item in enumerate(X[0]):
    if item.isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        encoder = preprocessing.LabelEncoder()
        X_encoded[:, i] = encoder.fit_transform(X[:, i])

X_data = X_encoded[:, :-1].astype(int)
y_data = X_encoded[:, -1].astype(int)

# 3. Розбиття на навчальну та тестову вибірки (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_data, y_data, test_size=0.2, random_state=5
)

# 4. Навчання SVM з Поліноміальним ядром (poly, degree=3)
# Використано degree=3 для забезпечення збіжності за розумний час
classifier = SVC(kernel='poly', degree=3, random_state=0)
classifier.fit(X_train, y_train)

# 5. Розрахунок показників якості
y_test_pred = classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_test_pred)
precision = precision_score(y_test, y_test_pred, average='weighted')
recall = recall_score(y_test, y_test_pred, average='weighted')
f1 = f1_score(y_test, y_test_pred, average='weighted')

print("=== Поліноміальне ядро (Polynomial Kernel) ===")
print(f"Акуратність (Accuracy):  {round(accuracy * 100, 2)}%")
print(f"Точність (Precision):    {round(precision * 100, 2)}%")
print(f"Повнота (Recall):        {round(recall * 100, 2)}%")
print(f"F1-міра (F1-score):      {round(f1 * 100, 2)}%")
```

LR_2_task_2_2.py:

```
import numpy as np
from sklearn import preprocessing
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Завантаження даних
input_file = 'income_data.txt'

X = []
y = []
count_class1 = 0
count_class2 = 0
max_datapoints = 25000

with open(input_file, 'r') as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break
        if '?' in line or len(line.strip()) == 0:
            continue
        data = line.strip().split(', ')
        if data[-1] == '<=50K' and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1
        elif data[-1] == '>50K' and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

X = np.array(X)

# 2. Кодування категоріальних ознак
label_encoder = []
X_encoded = np.empty(X.shape, dtype=object)

for i, item in enumerate(X[0]):
    if item.isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        encoder = preprocessing.LabelEncoder()
        X_encoded[:, i] = encoder.fit_transform(X[:, i])
```

```
        label_encoder.append(encoder)

X_data = X_encoded[:, :-1].astype(int)
y_data = X_encoded[:, -1].astype(int)

# 3. Розбиття на навчальну та тестову вибірки (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_data, y_data, test_size=0.2, random_state=5
)

# 4. Створення та навчання SVM з Гаусовим ядром (rbf)
classifier = SVC(kernel='rbf', random_state=0)
classifier.fit(X_train, y_train)

# 5. Прогнозування та розрахунок метрик
y_test_pred = classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_test_pred)
precision = precision_score(y_test, y_test_pred, average='weighted')
recall = recall_score(y_test, y_test_pred, average='weighted')
f1 = f1_score(y_test, y_test_pred, average='weighted')

print("=== Гаусове ядро (RBF Kernel) ===")
print(f"Акуратність (Accuracy):  {round(accuracy * 100, 2)}%")
print(f"Точність (Precision):    {round(precision * 100, 2)}%")
print(f"Повнота (Recall):        {round(recall * 100, 2)}%")
print(f"F1-міра (F1-score):      {round(f1 * 100, 2)}%")
```

LR_2_task_2_3.py:

```
import numpy as np
from sklearn import preprocessing
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Завантаження даних
input_file = 'income_data.txt'
```

```
y = []
count_class1 = 0
count_class2 = 0
max_datapoints = 25000

with open(input_file, 'r') as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break
        if '?' in line or len(line.strip()) == 0:
            continue
        data = line.strip().split(', ')
        if data[-1] == '<=50K' and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1
        elif data[-1] == '>50K' and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

X = np.array(X)

# 2. Кодування категоріальних ознак
label_encoder = []
X_encoded = np.empty(X.shape, dtype=object)

for i, item in enumerate(X[0]):
    if item.isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        encoder = preprocessing.LabelEncoder()
        X_encoded[:, i] = encoder.fit_transform(X[:, i])
        label_encoder.append(encoder)

X_data = X_encoded[:, :-1].astype(int)
y_data = X_encoded[:, -1].astype(int)

# 3. Розбиття на навчальну та тестову вибірки (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_data, y_data, test_size=0.2, random_state=5
```

```
# 4. Створення та навчання SVM з Сигмоїдальним ядром (sigmoid)
classifier = SVC(kernel='sigmoid', random_state=0)
classifier.fit(X_train, y_train)

# 5. Прогнозування та розрахунок метрик
y_test_pred = classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_test_pred)
precision = precision_score(y_test, y_test_pred, average='weighted')
recall = recall_score(y_test, y_test_pred, average='weighted')
f1 = f1_score(y_test, y_test_pred, average='weighted')

print("=== Сигмоїдальне ядро (Sigmoid Kernel) ===")
print(f"Акуратність (Accuracy):  {round(accuracy * 100, 2)}%")
print(f"Точність (Precision):    {round(precision * 100, 2)}%")
print(f"Повнота (Recall):        {round(recall * 100, 2)}%")
print(f"F1-міра (F1-score):      {round(f1 * 100, 2)}%")
```

Додаток В:

```
import numpy as np
import pandas as pd
from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# КРОК 1. ЗАВАНТАЖЕННЯ ТА ВИВЧЕННЯ ДАНИХ
iris_dataset = load_iris()

print("Ключі iris_dataset:\n", iris_dataset.keys())
print("\nКороткий опис (DESCR):\n", iris_dataset['DESCR'][:193] + "\n...")
print("\nНазви відповідей (target_names):\n", iris_dataset['target_names'])
```

```
print("Назви ознак (feature_names):\n", iris_dataset['feature_names'])
print("Тип масиву data:", type(iris_dataset['data']))
print("Форма масиву data:", iris_dataset['data'].shape)
print("Перші 5 елементів data:\n", iris_dataset['data'][:5])
print("Тип масиву target:{}".format(type(iris_dataset['target'])))
print("Масив target:\n", iris_dataset['target'])

# Завантаження через Pandas
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
try:
    dataset = pd.read_csv(url, names=names)
except Exception:
    # Резервний варіант з завантаженням через sklearn
    dataset = pd.DataFrame(iris_dataset.data, columns=names[:-1])
    dataset['class'] = [iris_dataset.target_names[i] for i in iris_dataset.target]

print("\nФорма датасету (shape):", dataset.shape)
print("\nПерші 20 рядків (head):\n", dataset.head(20))
print("\nСтатистичне зведення (describe):\n", dataset.describe())
print("\nРозподіл за класами:\n", dataset.groupby('class').size())

# КРОК 2. ВІЗУАЛІЗАЦІЯ ДАНИХ
# 1. Скриньки з вусами (Box Plot)
dataset.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
plt.suptitle('Boxplot of Input Features')
plt.tight_layout()
plt.show()

# 2. Гістограми
dataset.hist()
plt.suptitle('Histograms of Input Features')
plt.tight_layout()
plt.show()

# 3. Матриця діаграм розсіювання (Scatter Matrix)
scatter_matrix(dataset, figsize=(8,8))
plt.suptitle('Scatter Plot Matrix')
plt.tight_layout()
plt.show()
```

```
# КРОК 3. СТВОРЕННЯ НАВЧАЛЬНОГО ТА ТЕСТОВОГО НАБОРІВ
array = dataset.values
X = array[:, 0:4]
y = array[:, 4]

X_train, X_validation, Y_train, Y_validation = train_test_split(
    X, y, test_size=0.20, random_state=1
)

# КРОК 4. ПОРІВНЯННЯ 6 АЛГОРИТМІВ КЛАСИФІКАЦІЇ
models = []
models.append(('LR', LogisticRegression(solver='liblinear', multi_class='ovr')))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC(gamma='auto')))

results = []
names_models = []

print("\n--- Результати 10-кратної стратифікованої крос-валідації ---")
for name, model in models:
    kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring='accuracy')
    results.append(cv_results)
    names_models.append(name)
    print(f'{name}: {cv_results.mean():.6f} ({cv_results.std():.6f})')

# Графічне порівняння алгоритмів
plt.figure()
plt.boxplot(results, labels=names_models)
plt.title('Algorithm Comparison')
plt.ylabel('Accuracy')
plt.show()

# КРОК 6 & 7. ПРОГНОЗ ТА ОЦІНКА ЯКОСТІ НАЙКРАЩОЇ МОДЕЛІ (SVM /
KNN / LDA)
model = SVC(gamma='auto')
model.fit(X_train, Y_train)
predictions = model.predict(X_validation)
```

```
print("\n--- Оцінка точності на контрольні вибірці (Validation Set) ---")
print("Accuracy Score:", accuracy_score(Y_validation, predictions))
print("\nConfusion Matrix:\n", confusion_matrix(Y_validation, predictions))
print("\nClassification Report:\n", classification_report(Y_validation, predictions))

# КРОК 8. ПЕРЕДБАЧЕННЯ ДЛЯ НОВОЇ КВІТКИ
# Нова точка: довжина чашолистка=5, ширина чашолистка=2.9, довжина
пелюстки=1, ширина пелюстки=0.2
X_new = np.array([[5, 2.9, 1, 0.2]])
print("\nФорма масиву X_new:", X_new.shape)

prediction = model.predict(X_new)
print("Прогноз (клас):", prediction[0])
```

Додаток Г:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

input_file = 'income_data.txt'

X = []
y = []
count_class1 = 0
count_class2 = 0
max_datapoints = 10000  # Оптимальний розмір вибірки для швидких обчислень

with open(input_file, 'r') as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
```

```
            break
        if '?' in line or len(line.strip()) == 0:
            continue

        data = line.strip().split(', ')

        if data[-1] == '<=50K' and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1
        elif data[-1] == '>50K' and count_class2 < max_datapoints:
            X.append(data)
            count_class2 += 1

X = np.array(X)

# Перетворення категоріальних ознак на числові
X_encoded = np.empty(X.shape, dtype=object)
for i, item in enumerate(X[0]):
    if item.isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        encoder = preprocessing.LabelEncoder()
        X_encoded[:, i] = encoder.fit_transform(X[:, i])

X_data = X_encoded[:, :-1].astype(int)
y_data = X_encoded[:, -1].astype(int)

# Масштабування ознак для нормалізації роботи алгоритмів
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_data)

# Розбиття на навчальну та тестову вибірки (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_data, test_size=0.20, random_state=5
)

models = []
models.append(('LR', LogisticRegression(solver='liblinear')))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier(random_state=5)))
```

```
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC(gamma='auto', random_state=5)))

results = []
names = []

print("=== Результати 10-кратної стратифікованої крос-валідації (Accuracy) ===")
for name, model in models:
    kfold = StratifiedKFold(n_splits=10, random_state=1, shuffle=True)
    cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring='accuracy')
    results.append(cv_results)
    names.append(name)
    print(f"{name}: {cv_results.mean():.6f} ({cv_results.std():.6f})")

# Візуалізація порівняння алгоритмів (Boxplot)
plt.figure(figsize=(10, 6))
plt.boxplot(results, tick_labels=names)
plt.title('Порівняння алгоритмів класифікації на наборі даних Income')
plt.xlabel('Алгоритм')
plt.ylabel('Акуратність (Accuracy)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

print("\n" + "=" * 70)
print(f"{'Алгоритм':<10} | {'Accuracy (%)':<14} | {'Precision (%)':<14} | {'Recall
(%)':<12} | {'F1-Score (%)':<12}")
print("=" * 70)

metrics_summary = {}

for name, model in models:
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred) * 100
    prec = precision_score(y_test, y_pred, average='weighted') * 100
    rec = recall_score(y_test, y_pred, average='weighted') * 100
    f1 = f1_score(y_test, y_pred, average='weighted') * 100

    metrics_summary[name] = (acc, prec, rec, f1)
    print(f"{name:<10} | {acc:<14.2f} | {prec:<14.2f} | {rec:<12.2f} | {f1:<12.2f}")
```

```
print("=" * 70)
```

Додаток Д:

```
from io import BytesIO
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn import metrics
from sklearn.datasets import load_iris
from sklearn.linear_model import RidgeClassifier
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

iris = load_iris()
X, y = iris.data, iris.target

Xtrain, Xtest, ytrain, ytest = train_test_split(
    X, y, test_size=0.3, random_state=0
)

clf = RidgeClassifier(tol=1e-2, solver='sag', random_state=0)
clf.fit(Xtrain, ytrain)

ypred = clf.predict(Xtest)

print('--- Показники якості класифікатора Ridge ---')
print('Accuracy:', np.round(metrics.accuracy_score(ytest, ypred), 4))
print(
    'Precision:',
    np.round(metrics.precision_score(ytest, ypred, average='weighted'), 4),
)
print(
    'Recall:',
    np.round(metrics.recall_score(ytest, ypred, average='weighted'), 4),
)
print(
    'F1 Score:',
    np.round(metrics.f1_score(ytest, ypred, average='weighted'), 4),
)
print(
```

```
    'Cohen Kappa Score:', np.round(metrics.cohen_kappa_score(ytest, ypred), 4)
)
print(
    'Matthews Corrcoef:', np.round(metrics.matthews_corrcoef(ytest, ypred), 4)
)

print('\nClassification Report:\n')
print(
    metrics.classification_report(
        ytest, ypred, target_names=iris.target_names
    )
)

sns.set_theme()
mat = confusion_matrix(ytest, ypred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    mat.T,
    square=True,
    annot=True,
    fmt='d',
    cbar=False,
    xticklabels=iris.target_names,
    yticklabels=iris.target_names,
)
plt.xlabel('True label (Істинні класи)')
plt.ylabel('Predicted label (Передбачені класи)')
plt.title('Confusion Matrix for Ridge Classifier')
plt.tight_layout()

plt.savefig('Confusion.jpg')

f = BytesIO()
plt.savefig(f, format='svg')

plt.show()
```